

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

ĐHQG-HCM KHOA CÔNG NGHỆ THÔNG TIN



Môn học: Nhập môn Trí tuệ Nhân tạo

Lớp: 24CLC03

BÁO CÁO ĐỒ ÁN LAB 1

FloodRoute HCMC

Sinh viên

Ngô Quang Thắng 23127473

Vũ Lê Trọng Văn 20127095

Nguyễn Duy Khang 23127202

Lưu Huy Minh Quang 23127016

Giảng viên

TS. Nguyễn Văn Tân

Contents

1	Giới thiệu nhóm và phạm vi đồ án	1
1.1	Bối cảnh và mục tiêu	1
1.2	Thành viên và phân công	1
1.3	Phạm vi	1
1.4	Mapping yêu cầu - triển khai	1
2	Bối cảnh giao thông Việt Nam	1
2.1	Bài toán thực tế	1
2.2	Các yếu tố ảnh hưởng	1
2.3	Kịch bản shipper	2
2.4	Vì sao không chỉ tối ưu khoảng cách	2
3	Mô hình hóa bài toán	2
3.1	Graph có hướng	2
3.2	Thuộc tính edge	2
3.3	Composite cost	2
3.4	Ba cost profiles	2
4	Dataset và provenance	3
4.1	Dataset đang sử dụng	3
4.2	Nguồn dữ liệu và pipeline	3
4.3	Fixture kiểm chứng	3
5	Nguyên lý các thuật toán	3
6	Heuristic A* lower-bound và kiểm tra tính đúng đắn	4
6.1	Cách tính heuristic	4
6.2	Giải thích tính đúng đắn	4
6.3	Ảnh hưởng của scenario và weight	4
6.4	Phân biệt với Nearest Neighbor	4
7	Kiến trúc hệ thống và luồng thực thi	5
7.1	Kiến trúc	5
7.2	Giao diện chạy thực tế	5
7.3	Các luồng chính	6
7.4	Tính tái lập	6
8	Thực nghiệm và so sánh	6
8.1	Benchmark tìm kiếm trên bản đồ 65 node	6
8.2	Kiểm tra heuristic A* với UCS	6
8.3	Ảnh hưởng của cost profile	6
8.4	Ảnh chụp kết quả chạy thuật toán	7
9	Tối ưu tuyến giao hàng nhiều điểm	7
9.1	Định nghĩa bài toán	7
9.2	Held-Karp	11
9.3	Nearest Neighbor	11

9.4	Cách đọc kết quả	11
10	Cài đặt, giới hạn và hướng phát triển	11
10.1	Cách chạy	11
10.2	Giới hạn của đồ án	11
10.3	Hướng phát triển	11
10.4	Kết luận	12

1 Giới thiệu nhóm và phạm vi đồ án

1.1 Bối cảnh và mục tiêu

FloodRoute HCMC là đồ án Lab 1 của nhóm em trong môn Nhập môn Trí tuệ Nhân tạo. Nhóm em xây dựng một mô hình tìm đường giao hàng trên mạng đường có hướng, trong đó chi phí còn phụ thuộc vào thời gian, tình trạng traffic và nguy cơ ngập. Mục tiêu của đồ án là cài đặt các thuật toán tìm kiếm, giải thích được cách hoạt động và kiểm tra kết quả với UCS.

1.2 Thành viên và phân công

Thành viên	Mã số	Phạm vi chính
Ngô Quang Thắng	23127473	Mô hình graph, cost engine, tích hợp backend
Vũ Lê Trọng Văn	20127095	Các thuật toán tìm kiếm và trace frontier
Nguyễn Duy Khang	23127202	Dataset, scenario và kiểm thử
Lưu Huy Minh Quang	23127016	Frontend, bản đồ và báo cáo

1.3 Phạm vi

Trong phạm vi báo cáo này, nhóm em sử dụng bản đồ chính gồm 65 landmark và 178 directed road-path edges. Phần kiểm tra các tình huống peak/rain dùng một graph nhỏ có nhãn SIMULATED. Graph này chỉ phục vụ kiểm thử, không đại diện cho dữ liệu giao thông thời gian thực.

1.4 Mapping yêu cầu - triển khai

Yêu cầu Lab 1	Thành phần tương ứng
Tìm kiếm trên graph	BFS, DFS, UCS, A*, Greedy và Bidirectional Search
Tối ưu chi phí đa yếu tố	ScenarioCostEngine và ba cost profiles
Heuristic có chứng minh	lower-bound heuristic ρ_s của Chương 6
Nhiều điểm giao hàng	PairwiseMatrix, Nearest Neighbor và Held-Karp
Đánh giá thực nghiệm	benchmark search, lower-bound và cost profiles

2 Bối cảnh giao thông Việt Nam

2.1 Bài toán thực tế

TP. Hồ Chí Minh, đặc biệt là khu vực TP. Thủ Đức, thường có mật độ giao thông cao và nhiều tuyến đường một chiều. Vì vậy, tuyến đường ngắn nhất về khoảng cách chưa chắc là tuyến nhanh hoặc an toàn nhất khi traffic tăng, đường bị đóng hoặc xuất hiện ngập cục bộ.

2.2 Các yếu tố ảnh hưởng

Mỗi đoạn đường trong mô hình có hướng đi, thời gian khi đường thông thoáng, mức phạt do traffic, nguy cơ ngập và trạng thái đóng/mở. Nếu một đoạn đường bị đóng thì thuật toán không được phép đi qua đoạn đó. Traffic và flood được xem là các yếu tố làm thay đổi chi phí tùy theo từng scenario.

2.3 Kịch bản shipper

Trong kịch bản đặt ra, shipper xuất phát từ depot và giao hàng đến một hoặc nhiều landmark. Với một cặp start-goal, nhóm em cần tìm tuyến có tổng chi phí nhỏ nhất. Khi có nhiều điểm giao hàng, ngoài việc tìm đường giữa hai điểm còn phải quyết định thứ tự ghé thăm. Phần này được xem như một bài toán TSP delivery và trình bày ở Chương 9.

2.4 Vì sao không chỉ tối ưu khoảng cách

Khoảng cách Haversine chỉ phản ánh khoảng cách địa lý, không thể hiện hướng đường, mức độ tắc nghẽn hay nguy cơ ngập. Vì vậy, nhóm em tối ưu theo tổng composite cost. Heuristic A* chỉ dùng một lower bound an toàn để vẫn giữ được tính tối ưu của thuật toán.

3 Mô hình hóa bài toán

3.1 Graph có hướng

Nhóm em biểu diễn bản đồ dưới dạng graph có hướng $G = (V, E)$, trong đó node là các landmark và edge là các đoạn đường có thể đi qua. Mỗi node có mã, tên và tọa độ địa lý. Mỗi edge lưu node đầu, node cuối cùng các thông tin liên quan đến đoạn đường. Trong quá trình tìm kiếm, trạng thái của một node gồm chi phí tạm thời $g(n)$, node cha và trạng thái đã được khám phá hay chưa.

3.2 Thuộc tính edge

Một edge có các thuộc tính: distance, free-flow time, traffic penalty, flood risk, direction và closure status. Transition chỉ được phép trên edge không đóng. Graph được giữ bất biến trong suốt quá trình chạy. Các thuật toán chỉ đọc graph, không tự ý thay đổi dữ liệu gốc và sử dụng quy tắc tie-break cố định để kết quả có thể lặp lại.

3.3 Composite cost

Với scenario s , chi phí của một edge được tính bởi

$$\text{Cost}_s(e) = w_d d(e) + w_t t(e) + w_c c_s(e) + w_f r_s(e).$$

Trong đó d là distance, t là free-flow time, c_s là traffic penalty và r_s là flood risk. Các weight trong đồ án là hệ số ưu tiên dùng cho prototype và được chọn theo hướng mô phỏng/empirical. Đây không phải là phần trăm đóng góp thực tế, vì các thành phần có đơn vị và thang đo khác nhau.

3.4 Ba cost profiles

Profile	(w_d, w_t, w_c, w_f)
BALANCED	(0.25, 0.30, 0.20, 0.25)
PEAK_TRAFFIC	(0.15, 0.25, 0.45, 0.15)
RAIN_SAFE	(0.10, 0.25, 0.15, 0.50)

Các profile có thể được thay bằng API custom weights nếu có validation; closure vẫn là hard constraint.

4 Dataset và provenance

4.1 Dataset đang sử dụng

Trong phiên bản hiện tại, ứng dụng sử dụng release `thu_duc_landmarks_v1.0.0`, gồm 65 selectable landmarks và 178 directed road-path edges. Đây là bản đồ được dùng trong giao diện và benchmark search. Map 90 node đã được loại khỏi phạm vi sử dụng và không còn được dùng trong ứng dụng.

4.2 Nguồn dữ liệu và pipeline

Nhóm em tổ chức pipeline theo các bước:

`data/raw` $\longrightarrow$ `processed` $\longrightarrow$ `graph/scenario` $\longrightarrow$ `search`.

Nguồn đường và landmark được ghi nhận từ OSM/UTraffic; các trường được tính thêm trong quá trình xử lý được đánh dấu là derived. Nhóm em không chỉnh sửa trực tiếp `data/raw/`; các bản `processed` và `manifest` được version hóa để có thể kiểm tra lại.

Nhãn	Ý nghĩa
SOURCE_BACKED	Dữ liệu có nguồn bên ngoài được ghi nhận.
DERIVED	Dữ liệu được tính từ nguồn bằng pipeline hoặc công thức.
SIMULATED	Giá trị được tạo ra để chạy fixture hoặc scenario kiểm thử.
ASSUMPTION	Giả định tạm thời, cần được thay bằng dữ liệu thực khi có.

4.3 Fixture kiểm chứng

`toy_graph_v0.1` là graph nhỏ có nhãn SIMULATED, được dùng để kiểm tra các tình huống traffic/flood và lower-bound heuristic. Fixture này giúp nhóm em có các test case cố định, dễ chạy lại và không làm thay đổi dataset chính.

5 Nguyên lý các thuật toán

Trong đồ án, nhóm em cài đặt và so sánh các thuật toán sau:

Thuật toán	Ý tưởng	Tối ưu?	Độ phức tạp khái quát
BFS	Mở rộng các node theo từng lớp	Có nếu các edge có cùng cost	$O(V + E)$
DFS	Đi sâu theo một nhánh trước	Không đảm bảo	$O(V + E)$
UCS/Dijkstra	Luôn chọn node có g nhỏ nhất	Có với cost không âm	$O((V+E) \log V)$
A*	Chọn node theo $g + h$	Có nếu h admissible/consistent	Phụ thuộc heuristic
Greedy Best-First	Chọn node có h nhỏ nhất	Không đảm bảo	Phụ thuộc heuristic
Bidirectional	Tìm kiếm đồng thời từ hai đầu	Có nếu điều kiện dừng đúng	Giảm không gian trong một số trường hợp
Held-Karp	Dynamic programming trên tập stop	Có trong TSP nhỏ	$O(n^2 2^n)$
Nearest Neighbor	Mỗi bước chọn điểm gần/rẻ nhất tiếp theo	Không đảm bảo	$O(n^2)$

BFS phù hợp khi các edge có cùng chi phí. UCS được nhóm em dùng làm mốc tham chiếu vì luôn cho nghiệm tối ưu khi chi phí không âm. A* có thể giảm số node phải mở rộng nếu heuristic phù hợp. Greedy chạy nhanh

nhưng không thể dùng để khẳng định nghiệm tối ưu. Bidirectional có thể giúp giảm số node trong những graph phù hợp. Held-Karp cho nghiệm chính xác nhưng chỉ nên dùng khi số stop nhỏ, còn Nearest Neighbor cho nghiệm gần đúng với thời gian chạy ngắn hơn.

6 Heuristic A* lower-bound và kiểm tra tính đúng đắn

6.1 Cách tính heuristic

Với mỗi scenario s , nhóm em tính ρ_s một lần trước khi bắt đầu tìm kiếm:

$$\rho_s = \min_{\substack{e \in E, e \text{ không đóng} \\ \text{Haversine}(e) > 10^{-6} \text{ km}}} \frac{\text{Cost}_s(e)}{\text{Haversine}(e)}.$$

Sau đó, heuristic của một node n được tính như sau:

$$h_s(n) = \rho_s \cdot \text{Haversine}(n, \text{goal}), \quad f(n) = g(n) + h_s(n).$$

Các edge đã đóng không được đưa vào phép tính ρ_s . Những edge có khoảng cách Haversine quá nhỏ cũng được bỏ qua để tránh chia cho một số gần bằng 0. Nếu không còn edge hợp lệ thì nhóm em đặt $\rho_s = 0$. Với node không có tọa độ, heuristic được đặt bằng 0.

6.2 Giải thích tính đúng đắn

Vì chi phí edge không âm và ρ_s không lớn hơn tỉ số giữa cost và khoảng cách của mọi edge hợp lệ, chi phí của một đường đi luôn lớn hơn hoặc bằng lower bound do heuristic ước lượng. Kết hợp với bất đẳng thức tam giác của Haversine, ta có:

$$h_s(n) \leq \text{cost}(\text{một đường đi từ } n \text{ đến goal}).$$

Do đó $h_s(n) \geq 0$ và heuristic là admissible. Khi đi qua edge $e = (u, v)$, ta cũng có:

$$h_s(u) \leq \text{Cost}_s(e) + h_s(v),$$

nên heuristic là consistent. Với chi phí edge không âm, A* sẽ trả về đường đi có cost tối ưu, giống như UCS.

6.3 Ảnh hưởng của scenario và weight

Scenario được giữ cố định trong một lần tìm kiếm. Vì vậy, ρ_s và chi phí edge không thay đổi giữa các bước mở rộng node. Nếu traffic, flood hoặc các weight thay đổi thì cần tính lại ρ_s . Heuristic này tối ưu cho composite cost đang chọn, chứ không tối ưu riêng cho khoảng cách hoặc thời gian.

6.4 Phân biệt với Nearest Neighbor

A* dùng heuristic lower-bound để tìm đường cho một chặng từ Start đến Goal. Nearest Neighbor dùng chi phí đường đi giữa các cặp điểm để quyết định điểm nào sẽ được ghé tiếp theo. Do Nearest Neighbor chọn thứ tự theo cách greedy, việc dùng A* trong từng chặng không làm cho toàn bộ tour trở thành nghiệm tối ưu. Held-Karp mới là phương pháp exact trong giới hạn số stop nhỏ.

7 Kiến trúc hệ thống và luồng thực thi

7.1 Kiến trúc

Nhóm em chia chương trình thành các phần tương đối độc lập. Graph contract và các thuật toán nằm ở phần core, không phụ thuộc trực tiếp vào FastAPI. ScenarioCostEngine chịu trách nhiệm tính chi phí edge và heuristic lower-bound. SearchService chạy thuật toán tìm kiếm và lưu trace. PairwiseMatrix tạo ma trận chi phí giữa các điểm, sau đó TourOptimizerService dùng ma trận này cho Held-Karp hoặc Nearest Neighbor. FastAPI cung cấp API, còn React/MapLibre hiển thị bản đồ và kết quả.

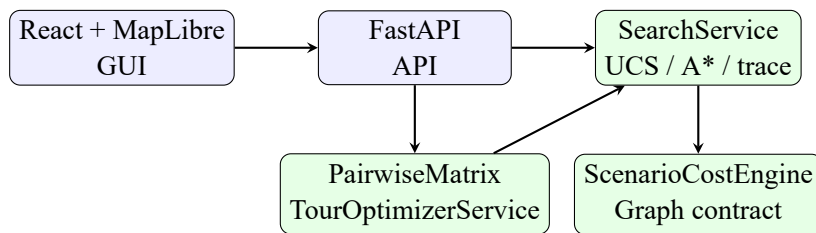


Figure 1: Sơ đồ các thành phần chính của FloodRoute HCMC.

7.2 Giao diện chạy thực tế

Figure 2 là ảnh chụp ứng dụng sau khi nhóm em chạy local và nạp bản đồ landmark. Bản đồ hiển thị 65 node và 178 directed road-path edges. Panel bên trái cho phép chọn dataset, thuật toán và scenario. Đây là ảnh chụp từ chương trình đang chạy, không phải hình dựng minh họa.

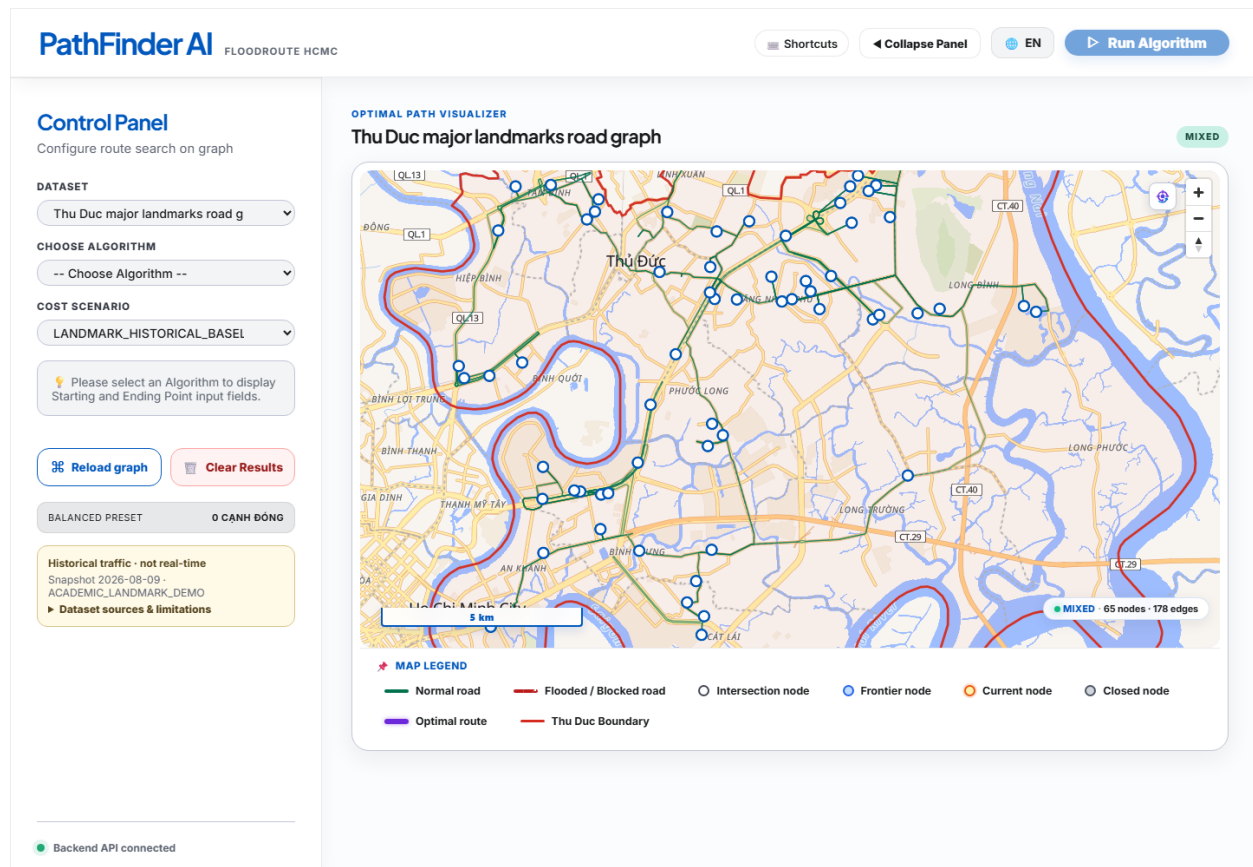


Figure 2: Giao diện FloodRoute HCMC khi nạp bản đồ active 65 node.

7.3 Các luồng chính

1. Tìm đường giữa hai điểm: nạp graph, giữ scenario cố định, tính ρ_s , chạy thuật toán và trả về path/cost.
2. So sánh thuật toán: chạy nhiều thuật toán trên cùng graph, start, goal và scenario.
3. Theo dõi trace: lưu frontier, current và closed để giao diện có thể phát lại quá trình tìm kiếm.
4. Tối ưu tour: tạo pairwise matrix rồi chạy Nearest Neighbor hoặc Held-Karp.
5. Mỗi chặng đường đều dùng UCS hoặc A* tùy lựa chọn, trong khi graph ban đầu vẫn được giữ nguyên.

7.4 Tính tái lập

Scenario, quy tắc tie-break và phiên bản bộ kết quả được lưu cùng kết quả chạy. Những giá trị mô phỏng được ghi nhãn SIMULATED; những giả định về dữ liệu thực được ghi nhãn ASSUMPTION.

8 Thực nghiệm và so sánh

8.1 Benchmark tìm kiếm trên bản đồ 65 node

Nhóm em chạy benchmark trên bản đồ `thu_duc_landmarks_v1.0.0` với 10 cặp OD, một baseline scenario, 6 thuật toán và 20 lần lặp cho mỗi trường hợp. Tổng cộng có 1,200 lượt chạy được đo. Các bảng và số liệu trong phần này được lấy từ file kết quả JSON sau khi chạy benchmark.

8.2 Kiểm tra heuristic A* với UCS

Để kiểm tra lower-bound heuristic trong nhiều điều kiện, nhóm em dùng fixture `toy_graph_v0.1` với 5 cặp OD và ba scenario: `OFFPEAK_BALANCED`, `PEAK_TRAFFIC`, `HEAVY_RAIN_SAFE`. Tổng cộng có 15 trường hợp.

Chỉ số	Kết quả
Cost mismatch A* so với UCS	0/15
Số case A* khám phá không quá UCS	15/15
Số scenario	3
Số cặp OD mỗi scenario	5

Kết quả cho thấy A* trả về cùng cost tối ưu với UCS trong cả 15 trường hợp. Số node A* khám phá cũng không lớn hơn UCS ở tất cả các case. Đây là kết quả kiểm thử cho phần cài đặt hiện tại, còn lý do về mặt lý thuyết đã được trình bày ở Chương 6.

8.3 Ảnh hưởng của cost profile

Ba profile `BALANCED`, `PEAK_TRAFFIC` và `RAIN_SAFE` sử dụng các hệ số được nêu ở Chương 3. Hai ví dụ trong kết quả chạy là:

- N01 → N06: baseline cost 2.343; peak cost 2.9209; rain cost 2.562625 và tuyến có thể phải đi đường vòng.
- N05 → N01: baseline cost 3.733 với path E14,E04,E02; rain cost 3.71325 với path E14,E10,E12,E02.

Các ví dụ này cho thấy khi thay đổi mức ưu tiên của traffic hoặc flood thì tuyến được chọn có thể thay đổi. Trong mọi profile, road closure vẫn được xử lý như một ràng buộc cứng.

8.4 Ảnh chụp kết quả chạy thuật toán

Các ảnh dưới đây được nhóm em chụp khi chạy ứng dụng trên máy với bản đồ 65 node. Ảnh thể hiện trực tiếp giao diện, trace và các chỉ số sau khi chạy, giúp người đọc dễ hình dung hơn về kết quả thực nghiệm.

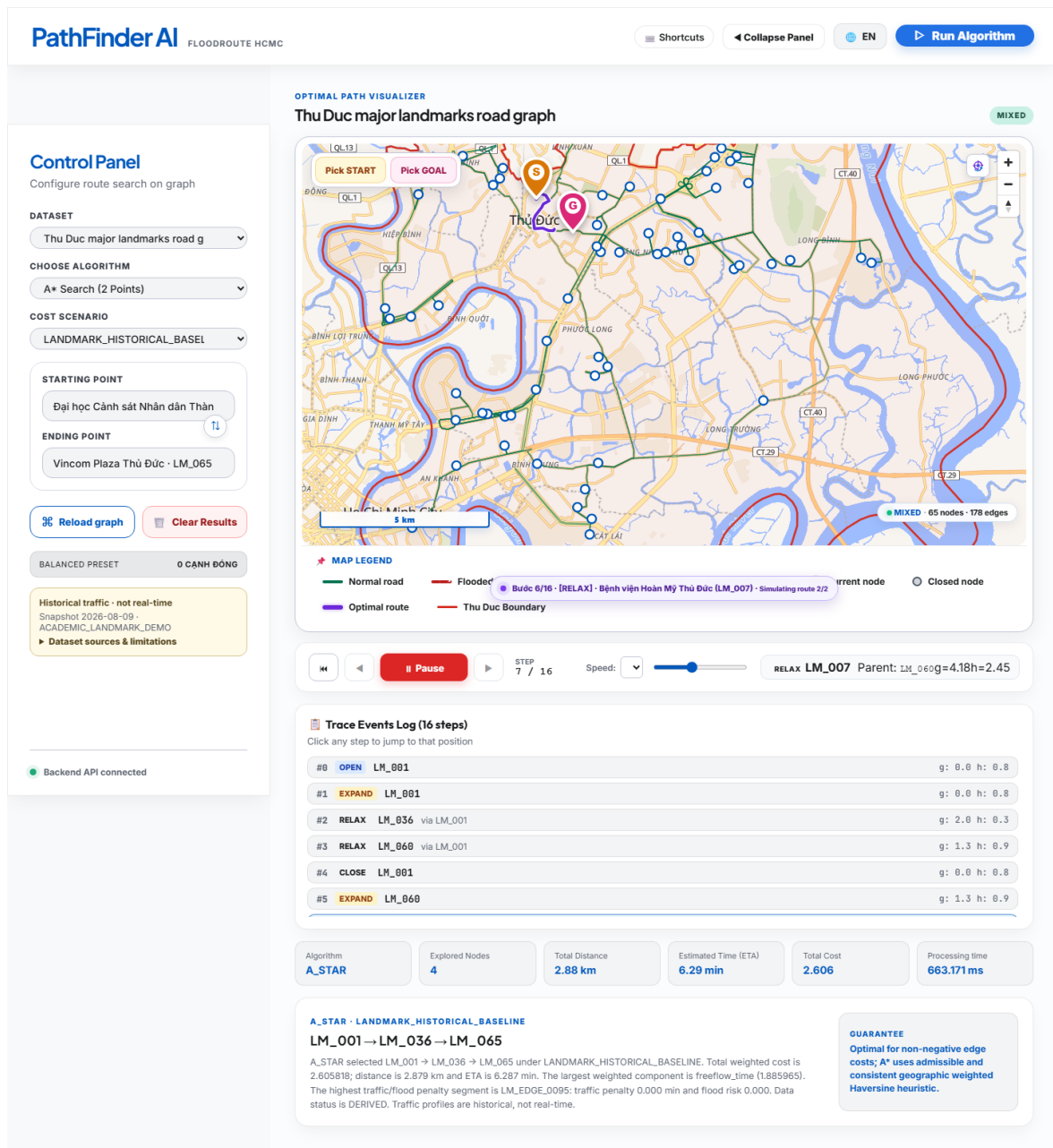


Figure 3: Kết quả chạy A* trên cặp LM_001 → LM_065, gồm route, trace và thông tin heuristic.

9 Tối ưu tuyến giao hàng nhiều điểm

9.1 Định nghĩa bài toán

Bài toán delivery gồm một depot và khoảng 5–10 điểm giao hàng. Với mỗi cặp điểm, nhóm em tính chi phí đường đi ngắn nhất để tạo thành pairwise matrix. Mục tiêu là tìm thứ tự đi qua các điểm sao cho tổng chi phí là nhỏ nhất. Tùy cấu hình, tuyến đường có thể quay lại depot.

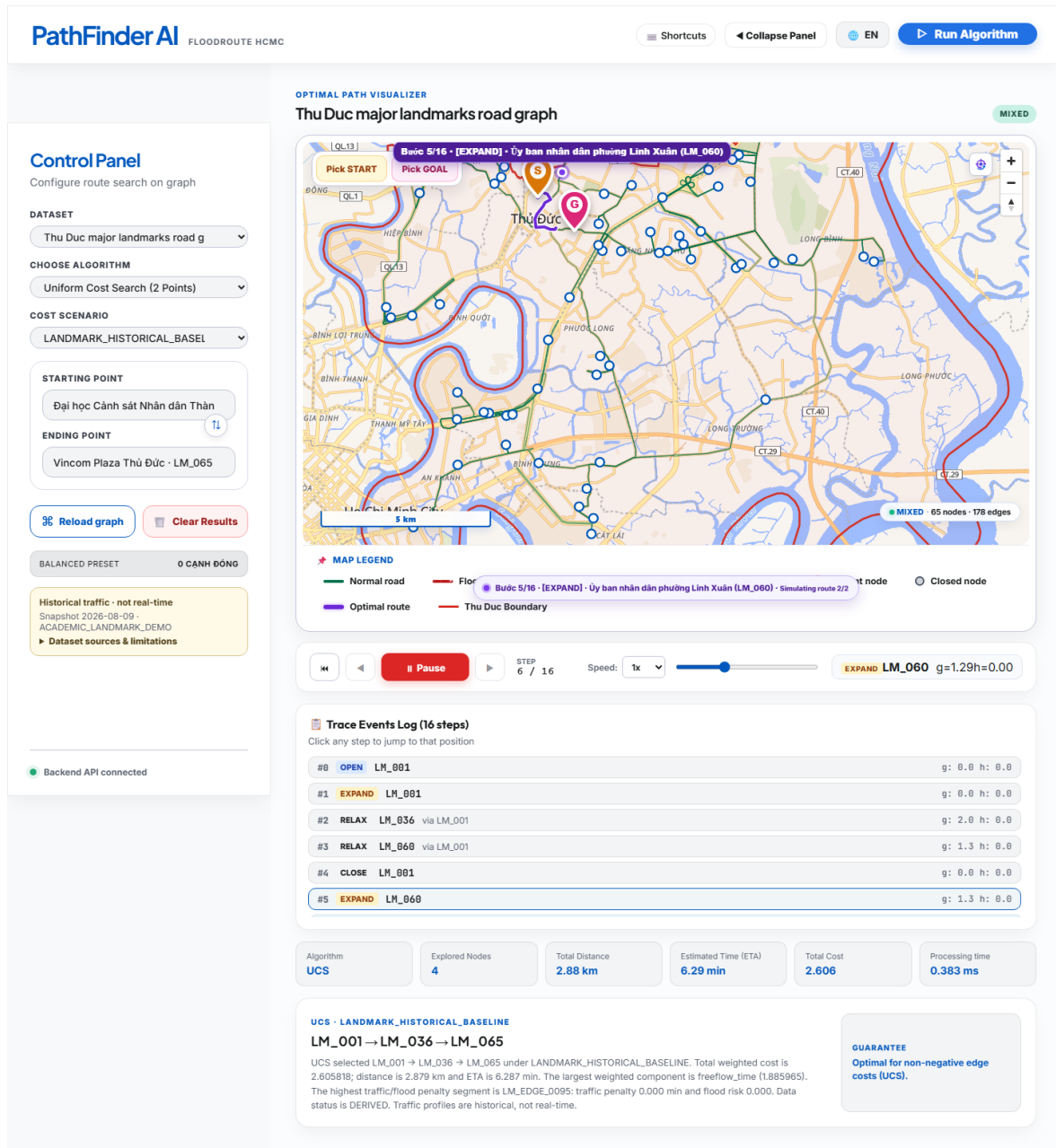


Figure 4: Kết quả chạy UCS trên cùng cặp start-goal, được dùng làm kết quả đối chiếu với A*.

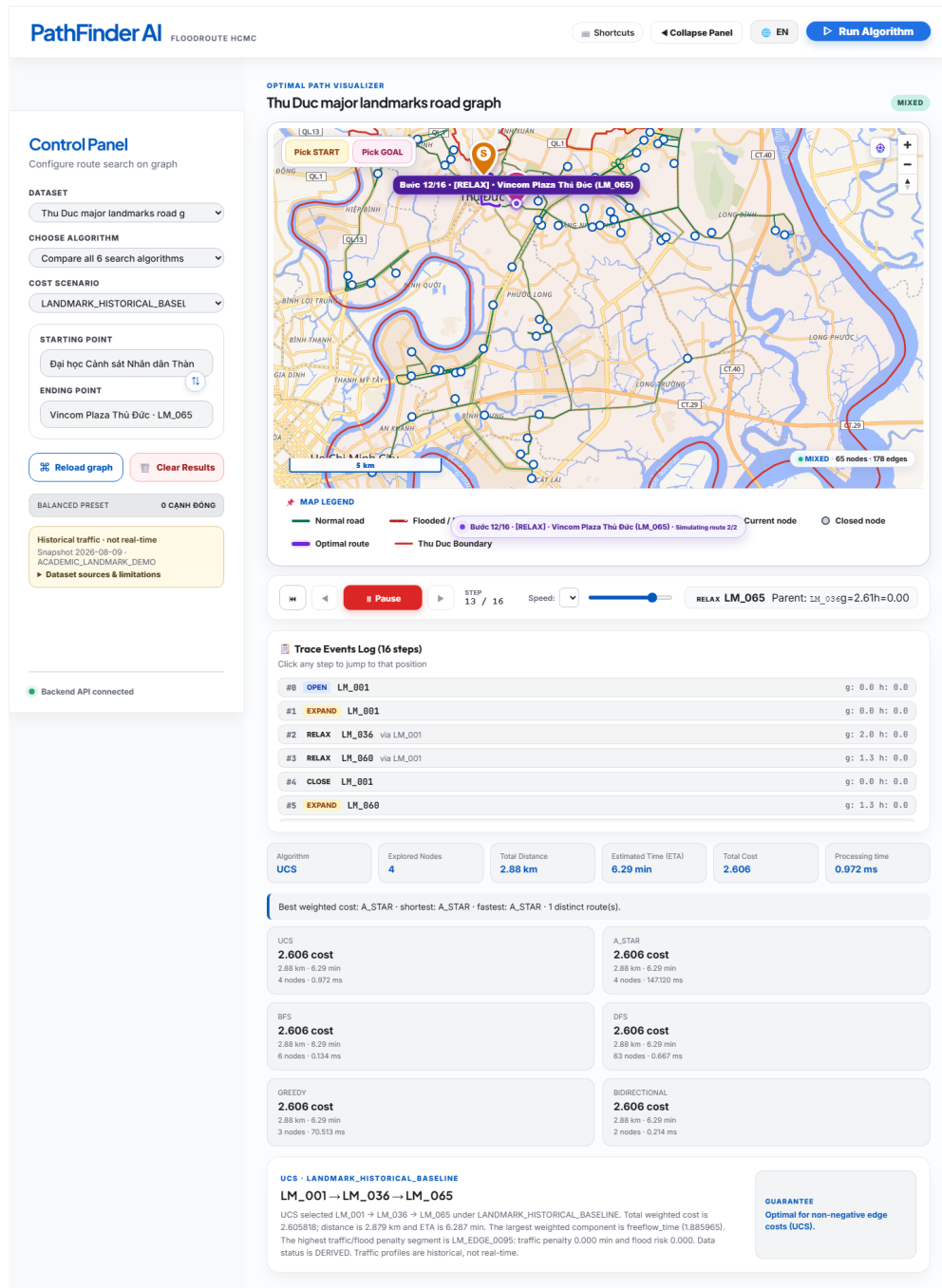
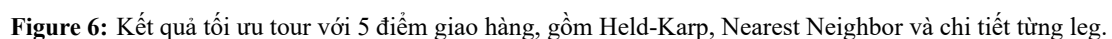


Figure 5: Chế độ Compare hiển thị cost và số node đã khám phá của 6 thuật toán.



9.2 Held-Karp

Held-Karp lưu kết quả theo tập các stop đã đi qua và điểm cuối cùng. Đây là phương pháp dynamic programming cho nghiệm chính xác, có độ phức tạp $O(n^2 2^n)$. Vì độ phức tạp tăng nhanh, nhóm em chỉ dùng Held-Karp làm nghiệm tham chiếu khi số stop còn nhỏ.

9.3 Nearest Neighbor

Nearest Neighbor bắt đầu từ depot. Ở mỗi bước, thuật toán chọn điểm chưa đi qua có pairwise cost nhỏ nhất. Khi có nhiều lựa chọn bằng nhau, node id được dùng để quyết định theo một thứ tự cố định. Cách này có độ phức tạp khoảng $O(n^2)$ sau khi đã có matrix, chạy nhanh nhưng chỉ cho nghiệm gần đúng.

9.4 Cách đọc kết quả

Khi đánh giá tour, nhóm em so sánh thứ tự ban đầu, kết quả của Nearest Neighbor và kết quả của Held-Karp dựa trên tổng cost, thứ tự stop và thời gian chạy. A* hoặc UCS chỉ giải quyết từng chặng riêng lẻ; chất lượng của cả tour còn phụ thuộc vào thuật toán chọn thứ tự điểm.

10 Cài đặt, giới hạn và hướng phát triển

10.1 Cách chạy

Sau khi cài đặt backend và frontend theo README, nhóm em dùng các lệnh sau để kiểm tra:

```
npm run test:data
npm run test:backend
npm run test:frontend
npm run benchmark:search
npm run benchmark:lower-bound
npm run benchmark:cost-profiles
```

Để biên dịch báo cáo, chạy:

```
cd docs/reports
xelatex -interaction=nonstopmode main.tex
xelatex -interaction=nonstopmode main.tex
```

10.2 Giới hạn của đồ án

Dataset hiện tại chưa phải dữ liệu traffic real-time. Một phần traffic/flood trong fixture là dữ liệu SIMULATED; một số nhận định cho dữ liệu tương lai vẫn là ASSUMPTION. Nếu cho phép người dùng nhập custom weights thì cần kiểm tra giá trị đầu vào và thang đo. Held-Karp có độ phức tạp tăng theo cấp số mũ nên chỉ phù hợp với số stop nhỏ. Heuristic lower-bound bảo đảm tính tối ưu cho composite cost hiện tại, nhưng có thể không mạnh nếu cost profile thay đổi nhiều.

10.3 Hướng phát triển

Trong tương lai, nhóm em có thể bổ sung traffic real-time, thêm scenario peak/rain cho landmark release, hỗ trợ custom weights có validation, bài toán nhiều xe và time windows. Giao diện cũng có thể được mở rộng để hiển thị nhiều thông tin hơn về cost và trace. Khi thêm các thành phần mới, cần cập nhật provenance, benchmark và các contract liên quan.

10.4 Kết luận

Qua đồ án, nhóm em xây dựng được một quy trình tìm đường gồm UCS, A*, các thuật toán tìm kiếm khác và hai phương pháp tối ưu tour. UCS được dùng làm mốc để kiểm tra A*. Trên 15 trường hợp kiểm thử lower-bound, A* cho cùng cost với UCS và không khám phá nhiều node hơn UCS. Kết quả này cho thấy heuristic đang dùng phù hợp với composite cost và các scenario đã kiểm tra.

References

- [1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson, 2020.
- [2] Michael Held and Richard M. Karp. A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [4] OpenStreetMap Contributors. Planet dump retrieved from <https://planet.openstreetmap.org>, 2026.
- [5] UTraffic Dataset: Urban Traffic Flow and Speed Profiling in Ho Chi Minh City. Kaggle Open Dataset, 2025.